

Persistencia de datos

Manejo de Archivos en C

¿Qué pasa con nuestros datos cuando el programa termina?

- Toda la información que manejamos (variables, arreglos, estructuras) se almacena en la Memoria RAM.
- La RAM es volátil: su contenido se borra cuando el dispositivo se apaga o el programa finaliza.
- Problema: ¿Cómo guardamos la configuración de un usuario, el puntaje más alto de un juego o un documento de texto?

El Almacenamiento Persistente



- Un **archivo** es una colección de datos almacenados en un medio secundario (disco duro, SSD, pendrive). Es el contenedor de nuestra información.
- Permite que la información sea **persistente**, es decir, que perdure incluso después de que el programa termine.
- Para simplificar la interacción, C nos provee una **abstracción** llamada **secuencia (stream)**. Es el **canal de comunicación** estandarizado entre nuestro programa y el archivo.

Archivos vs Secuencias

- El **sistema de E/S de C** está diseñado para ser **universal**: trata a los distintos dispositivos de manera consistente.
- El **Archivo** es el **dispositivo físico o el contenedor de datos real** (ej: un .txt en el disco, la impresora, la consola).
- La **Secuencia (Stream)** es la **abstracción lógica**: un canal de comunicación estandarizado que el programa utiliza para interactuar con el archivo.
- Gracias a las secuencias, nuestro código **trata de la misma manera** a un archivo de texto, la consola o una impresora.
- Existen **dos tipos de secuencias: de texto y binarias**.

Identificando un Archivo: La Ruta

- Para abrir un archivo, primero debemos decirle al sistema **cuál es y dónde está**.
- La **ruta (path)** es una cadena de texto (string) que actúa como la "**dirección**" **única** de un archivo en el sistema.
- **Ruta Absoluta:** Describe la ubicación completa desde la raíz del sistema.
 - Ejemplo: C:\Users\MiUsuario\Desktop\datos.txt (Windows) o /home/usuario/documentos/datos.txt (Linux).
- **Ruta Relativa:** Describe la ubicación desde el directorio actual donde se ejecuta el programa. Es más corta y portable.
 - Ejemplo: datos.txt (si está en la misma carpeta) o ../recursos/config.ini.
- **Permisos:** El sistema operativo controla quién puede leer o escribir un archivo. Una apertura puede fallar si no tenemos los permisos necesarios.

¿Cómo manejamos un archivo en C?

- C define un tipo de dato **struct** llamado **FILE** en la librería **<stdio.h>**.
- Esta estructura contiene toda la información necesaria para controlar una secuencia (el estado del archivo, la posición actual del cursor, etc.).
- No accedemos directamente a sus miembros. Interactuamos con el archivo a través de un puntero a esta estructura.

#include <stdio.h>

FILE *puntero_archivo;

Abriendo el Canal de Comunicación

- La función **fopen()** inicializa la conexión entre nuestro programa y un archivo físico.
- Recibe el nombre (o ruta) del archivo y el modo de apertura.
- Devuelve un puntero FILE * si tiene éxito.
- Devuelve NULL si falla (el archivo no existe, no hay permisos, etc.).

```
FILE *fp;  
fp = fopen("datos.txt", "w"); // "w" para escritura (write)  
if (fp == NULL) { // Se verifica que la apertura fue exitosa  
    printf("Error al abrir el archivo.\n");  
    return 1; // Terminar el programa con un código de error  
}
```

Ejemplo 1

¿Qué queremos hacer con el archivo?

Modo	Descripción	Si el archivo NO existe	Si el archivo SÍ existe
r	Lectura (Read). El puntero se posiciona al inicio.	Falla (devuelve `NULL`)	Se puede leer desde el inicio.
w	Escritura (write)	Se crea.	Se borra todo el contenido.
a	Añadir (append)	Se crea.	El puntero se posiciona al final.
r+	Lectura y Escritura	Falla (devuelve `NULL`)	Se puede leer y escribir.
w+	Lectura y Escritura	Se crea.	Se borra todo el contenido.
a+	Lectura y Añadir	Se crea.	Se puede leer. La escritura es al final.

Cerrando la Conexión

La función **fclose()** cierra la secuencia, terminando la conexión con el archivo. Es muy importante hacerlo porque:

- **Guarda los cambios:** El SO a menudo escribe en el disco en bloques (usando un buffer). “fclose” se asegura de que todo lo que quedaba en el buffer se escriba físicamente en el archivo.
- **Libera recursos:** SO tiene un límite de archivos que pueden estar abiertos a la vez.

fclose(fp); // Cierra el archivo asociado al puntero fp

fp = NULL; // Buena práctica para evitar usar un puntero ya cerrado

Ejemplo 1

```
#include <stdio.h>
int main() {
    FILE *fp;
    printf("Intentando abrir 'datos.txt' en modo escritura ('w')...\n");
    fp = fopen("datos.txt", "w");
    if (fp == NULL) {
        printf("Error al abrir el archivo");
        return 1;
    }
    printf("Archivo datos.txt abierto con exito\n");
    printf("Ahora, cerrando el archivo...\n");
    fclose(fp);
    fp = NULL;
    printf("Archivo cerrado.\n");
    return 0;
}
```

Entrada y Salida en C

Propósito	Función(es)
I/O de Caracteres	fgetc(), fputc()
I/O de Cadenas	fgets(), fputs()
I/O con Formato	fscanf(), fprintf()
I/O de Bloques (Structs)	fread(), fwrite()

Manejo de los datos en el archivo

Operaciones Carácter por Carácter

Son las funciones más básicas para interactuar con archivos de texto.

- **int fputc(int character, FILE *fp);**
 - Escribe un único carácter en el archivo.
- **int fgetc(FILE *fp);**
 - Lee un único carácter del archivo.
 - Avanza el indicador de posición del archivo al siguiente carácter.
 - Devuelve EOF (End Of File) cuando llega al final.

Ejemplo 2

```
#include <stdio.h>
int main() {
    FILE *in, *out;
    int ch; // Usamos int para poder comparar con EOF
    in = fopen("origen.txt", "r");
    out = fopen("destino.txt", "w");
    // Comprueba que no sean NULL *in y *out
    printf("Copiando 'origen.txt' a 'destino.txt'...\n");
    while ((ch = fgetc(in)) != EOF) {
        fputc(ch, out); // Escribimos el caracter leído en el archivo de destino.
    }
    printf("Copia completada.\n");
    fclose(in);
    fclose(out);
    return 0;
}
```

Entrada y Salida en C

Propósito	Función(es)
I/O de Caracteres	fgetc(), fputc()
I/O de Cadenas	fgets(), fputs()
I/O con Formato	fscanf(), fprintf()
I/O de Bloques (Structs)	fread(), fwrite()

Manejo de los datos en el archivo

Operaciones con Cadenas de Texto

Más eficientes que procesar carácter por carácter.

- **int fputs(const char *cadena, FILE *fp);**
 - Escribe una cadena de texto en el archivo. No añade el `\\n`.
- **char *fgets(char *buffer, int n, FILE *fp);**
 - Lee una línea de texto del archivo y la guarda en `buffer`.
 - Lee como máximo `n-1` caracteres o hasta encontrar un `\\n` (salto de línea).
 - A diferencia de `gets`, es segura porque limita el tamaño.
 - Devuelve `NULL` al llegar al final del archivo.

```
#include  
#define MAX_LINEA 100 // Tamaño del buffer  
int main() {  
    FILE *fp;  
  
    return 0;  
}
```

Ejemplo 3 `<stdio.h>`

Entrada y Salida en C

Propósito	Función(es)
I/O de Caracteres	fgetc(), fputc()
I/O de Cadenas	fgets(), fputs()
I/O con Formato	fscanf(), fprintf()
I/O de Bloques (Structs)	fread(), fwrite()

Manejo de los datos en el archivo

printf y scanf para Archivos

Permiten escribir y leer datos con un formato específico, ideal para archivos de texto estructurados.

- **int fprintf(FILE *fp, const char *formato, ...);**
 - Funciona exactamente como printf, pero escribe en un archivo en lugar de la consola.
- **int fscanf(FILE *fp, const char *formato, ...);**
 - Funciona como scanf, pero lee desde un archivo.
 - Devuelve el número de ítems leídos o EOF si llega al final.

// Para escribir

```
fprintf(fp, "%s %d %.2f\n", "Juan", 25, 180.5);
```

// Para leer

```
char nombre[50];
```

```
int edad;
```

```
float altura;
```

```
fscanf(fp, "%s %d %f", nombre, &edad, &altura);
```

Ejemplo
4 y 5

Entrada y Salida en C

Propósito	Función(es)
I/O de Caracteres	fgetc(), fputc()
I/O de Cadenas	fgets(), fputs()
I/O con Formato	fscanf(), fprintf()
I/O de Bloques (Structs)	fread(), fwrite()

Manejo de los datos en el archivo

¿Cómo guardamos nuestros struct?

Usar fprintf para estructuras con muchos campos es tedioso y propenso a errores.

Problema: ¿Cómo guardamos una estructura entera en un solo paso?

Solución: Escribiendo una copia exacta de los bytes que ocupa la estructura en memoria. Esto se hace en modo binario.

Modo Binario: Se añade una **b** al **modo de apertura** ("wb", "rb", "rb+", etc.).

- "wb": Escritura binaria (Write Binary).
- "rb": Lectura binaria (Read Binary).

Funciones fread() y fwrite()

Ejemplo 6

Estas funciones leen y escriben bloques de memoria de cualquier tipo de dato.

size_t fwrite(const void* ptr, size_t size, size_t n, FILE* fp);

- ptr: Puntero al dato a escribir (ej: &mi_structura).
- size: Tamaño en bytes de cada elemento (ej: sizeof(mi_structura)).
- n: Cantidad de elementos a escribir (usualmente 1 si es una sola estructura).
- fp: Puntero al archivo.

size_t fread(void* ptr, size_t size, size_t n, FILE* fp);

- Funciona de manera análoga para leer desde el archivo hacia nuestra variable.

Borrando Archivos

Ejemplo 7

- La función **remove()**, de **<stdio.h>**, se usa para eliminar un archivo del sistema de ficheros.
- Recibe como argumento el nombre o la **ruta del archivo**.
- Devuelve 0 si la eliminación fue exitosa, y un valor distinto de cero si hubo un error.

```
#include <stdio.h>
```

```
if (remove("archivo_a_borrar.txt") == 0) {  
    printf("Archivo eliminado exitosamente.\n");  
} else {  
    printf("Error: no se pudo eliminar el archivo.\n");  
}
```

Resumen de funciones

Propósito	Función(es)
Declarar puntero	FILE *fp;
Abrir / Cerrar	fopen(), fclose()
I/O de Caracteres	fgetc(), fputc()
I/O de Cadenas	fgets(), fputs()
I/O con Formato	fscanf(), fprintf()
I/O de Bloques (Structs)	fread(), fwrite()
Eliminar	remove()

Naveguemos en:
[https://conclase.net/c/
librerias](https://conclase.net/c/librerias)

Función feof()

Se usa para detectar si se ha alcanzado el final de un archivo.

Sintaxis:

int feof(FILE *stream);

stream: Puntero al archivo.

Valor de retorno

Devuelve 0 si NO se ha alcanzado el final del archivo.

Devuelve un valor distinto de 0 si se ha alcanzado el final.

Ejemplo Función feof()

```
while (!feof(archivo)) {  
    fgets(buffer, 100, archivo);  
    printf("%s", buffer);  
}
```

Lee un archivo línea por línea hasta llegar al final.

Evita errores de lectura inesperados.

Función feof()

La función **feof()** detecta el final de todo el archivo (EOF, "End of File"), no el final de una línea.

El EOF se alcanza cuando se ha leído todo el contenido del archivo. Esto ocurre cuando **el puntero de lectura llega al final físico del archivo**, es decir, después del último byte de datos.

No es lo mismo que...

Carácter	Nombre	Uso Principal	¿Dónde se encuentra?	Significado
\n	Salto de Línea (Newline)	Separar líneas de texto	Dentro de los archivos de texto	"Fin de la línea actual, la siguiente información comienza abajo"
\0	Terminador Nulo (Null)	Marcar el final de un string	En la memoria RAM, al final de un arreglo de char	"Este es el fin de mi cadena de caracteres"